안녕하세요, 우정현입니다.

사용자가 더 많이 쓰게 만드는 방법을, 빨리 만들고 반응을 보며 찾는 개발자입니다.
commit3921@gmail.com / 010-5127-3921

빨리 만들어 반응부터 봅니다.

소크라틱 런은 CLI 학습 도구를 웹으로 옮기며 학습 흐름 7단계를 GA4로 계측했고, 책췍 실험 페이지는 두 책의 읽음 여부·페이지 체류·유입 경로를 기록하도록 만들었습니다. 링구아는 앱 설치에 대한 거부감이 커서 웹으로 옮겨 새로 실험하고 있습니다. 셋 다 에이전틱 코딩으로 빠르게 만들어, 완성도보다 사용자 반응부터 확인했습니다.

결과가 기대와 다르면 내 방법부터 의심합니다.

책췍 1차 실험에서 조회에 비해 참여율이 낮게 나오자, 실험 방법·데이터·배포 채널 중 어디가 틀렸는지 쪼갰습니다. '감상을 쓸 맥락이 부족했다'는 가설로 2차 실험을 설계했고, 이후 실험들도 이어서 설계하고 있습니다.

사용자를 막는 것은 숫자로 확인하고 걷어냅니다.

학습 서비스에서 로그인 게이트가 있던 기간에 학습을 시작한 방문자가 11.5%에 그친 것을 보고, 호출 상한으로 비용을 막는 대신 로그인을 없앴습니다. 이후 진입률은 26.7%로 올랐습니다.

주요 기술	Kotlin / Jetpack Compose / Compose Multiplatform Flutter / Dart / Firebase
언어	TOEIC 940 (최고 965) / 일본어 OPIc IM2

경력

2025.09 - 2026.02	네모반듯 / 툭 외주 앱 인수 후 1인 개발·운영
2024.01 - 2025.08	뉴지니 / 비드톡톡 전자입찰 앱을 협력사 브랜드 16개로 배포
2023.09 - 2023.12	모바일 앱 개발 협동조합 / iCare 안 쓰던 스마트폰을 홈 카메라로 쓰는 앱, 기존 TFLite 모델로 넘어짐과 울음 감지

교육

2026.02 -	우아한테크코스 Android 8기 영남대학교 컴퓨터공학과 졸업

Link

github.com/CommitTheKermit
socratic-learn-web.web.app
play.google.com/store/apps/details?id=kr.co.newjini.bidtoktok

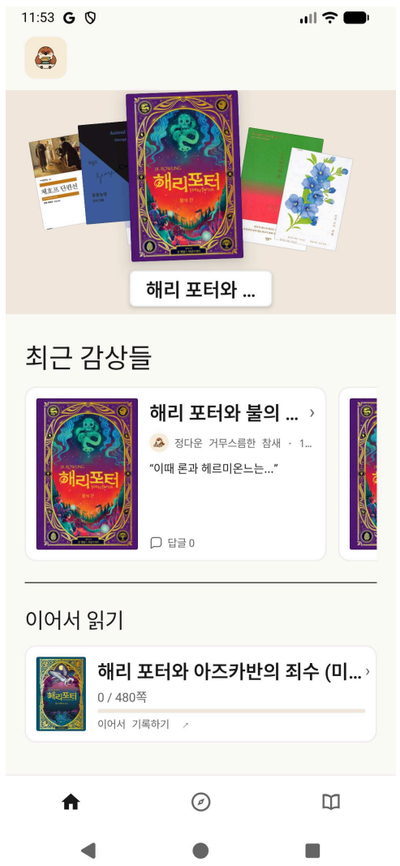
책췍 github.com/woowacourse-teams/2026-chaekchaek

왜 감상을 안 남길까, 실험으로 좁혀 가기

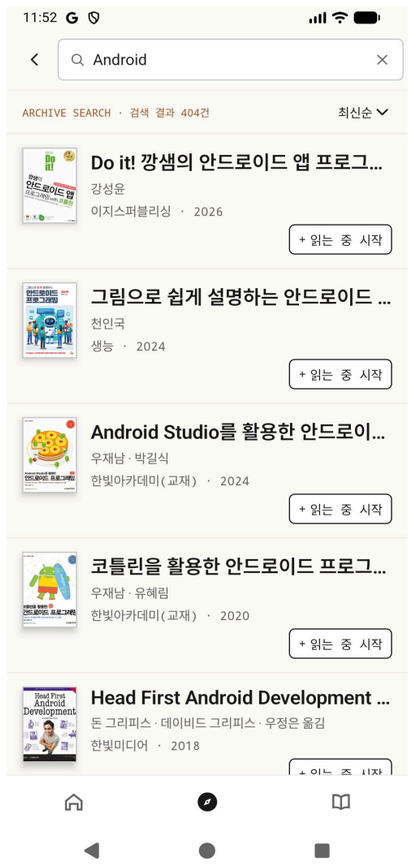
우아한테크코스 / 2026.08 - 진행 중
Android 개발, 사용자 실험 설계·구현

책췍은 읽은 책의 감상을 익명 또는 닉네임으로 남기고 다른 사람과 나누는 서비스입니다.

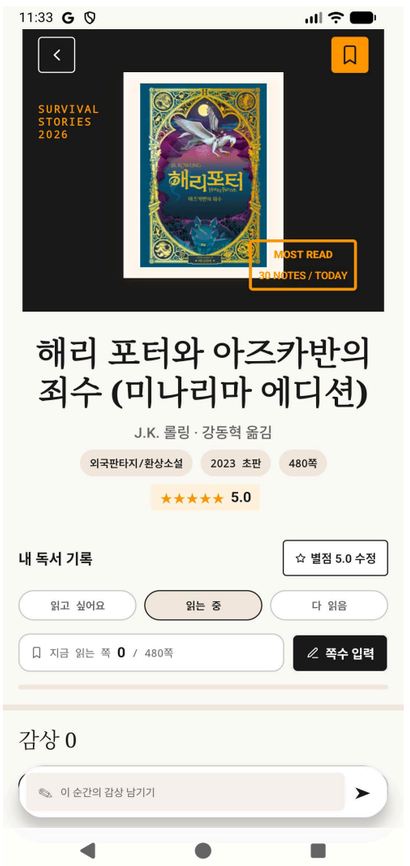
앱을 만드는 것과 함께, 사람들이 어떤 조건에서 감상을 남기는지 확인하는 웹 실험을 설계하고 만들었습니다. 결과가 기대와 다를 때마다 사용자보다 제 실험부터 의심하며 문제를 좁혀 갔습니다.



캡션 입력



캡션 입력



캡션 입력



캡션 입력

1차 실험

조회는 많았지만 감상은 적었음

어떤 장르에서 감상을 나누고 싶어 하는지 보려고 Codex로 하루 안에 실험 페이지를 만들어 여러 장르의 책을 올렸습니다. 조회는 많았지만 감상을 남기지 않거나 감상으로 보기 어려운 댓글이 달리는 등 참여율이 낮았습니다.

원인 좁히기

사용자보다 내 실험부터 의심

책이 부적절 했는지, 장르에 관심이 없던 건지, 쓰기 어려웠던 건지 구분할 수 없었습니다. 실험 방법, 데이터, 배포 채널을 하나씩 따져 보고 팀 회의, 피드백으로 문제를 '참여가 적다'에서 '감상을 쓸 맥락이 부족했다'로 좁혔습니다.

2차 실험

책 두 권, 발췌문과 예시 감상, 읽음 여부 분리

장르를 넓게 보여주는 대신 책 두 권에 집중해 발췌문 6쪽과 먼저 남겨진 감상을 제공하고, 읽은 사람과 안 읽은 사람을 나눠 비교했습니다. 책별로 발췌 페이지 열람과 화면 노출 시간, 작성 시작과 제출, 유입 경로(UTM)와 기기를 기록했고, 방문은 읽음 여부를 고른 책에만 집계해 두 책의 수치가 섞이지 않게 했습니다.

결과

안 읽은 사람은 55명 중 1명만 감상을 남김

읽은 사람은 30명 중 3명이 감상을 남겼지만, 안 읽은 사람은 55명 중 1명에 그쳤고 발췌문도 절반 넘게 1쪽에서 멈췄습니다(이방인 56명 → 2쪽 22명). 맥락을 주는 것만으로는 안 읽은 사람을 참여로 이끌지 못했습니다. 에브리타임에서는 책 커뮤니티에 실험 링크를 배포하였고 대부분 '읽었어요', 슬랙·링크드인 방문자는 대부분 '안 읽었어요'를 골라 배포 채널이 참여자 구성을 바꾼다는 것도 보였습니다.

다음

참여 부담을 낮추고, 특화 커뮤니티와 책 추천으로 실험 확장

책 커뮤니티에서 '읽었어요' 비율이 높았던 만큼 특화 커뮤니티 홍보를 늘리려 합니다. 감상 작성 대신 O/X로 답하거나 '당신이 뫼르소였다면?'처럼 상황을 묻는 방식으로 참여 부담을 낮추는 실험과, 인스타그램 홍보 콘텐츠 중 참여율이 높았던 '책 추천'을 활용한 실험을 계획하고 있습니다.

사용 기술

앱 : Kotlin / Compose Multiplatform, 실험 웹 : Next.js / Supabase / Vercel

---

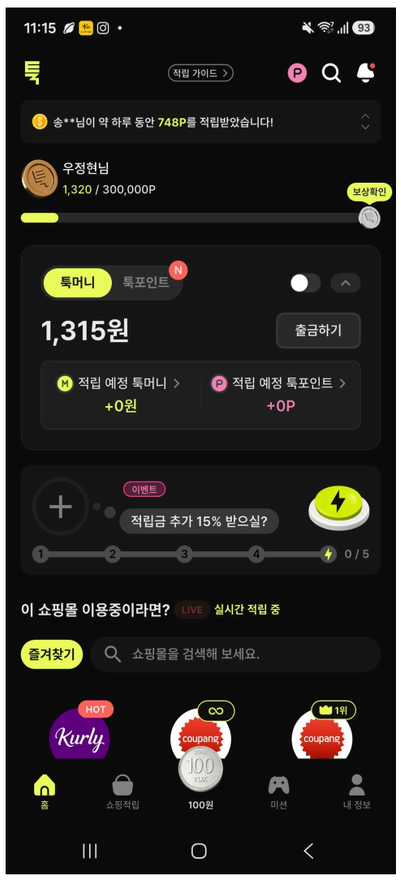
툭 네모반듯 리워드 커머스 앱

인수한 앱의 발열을 잡고, 새는 보상을 막기

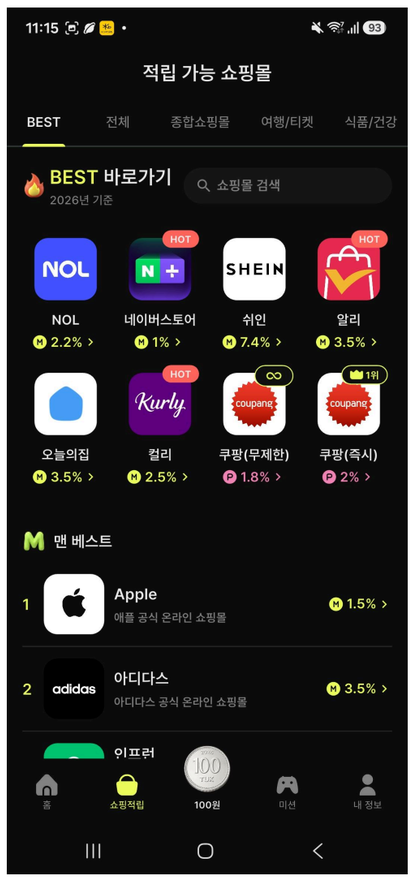
네모반듯 / 2025.09 - 2026.02
외주 개발본 인수 후 1인 개발

툭은 제휴 쇼핑몰에서 구매하면 캐시백을, 미션과 참여형 콘텐츠로 포인트를 적립해 출금하는 리워드 커머스 앱입니다.

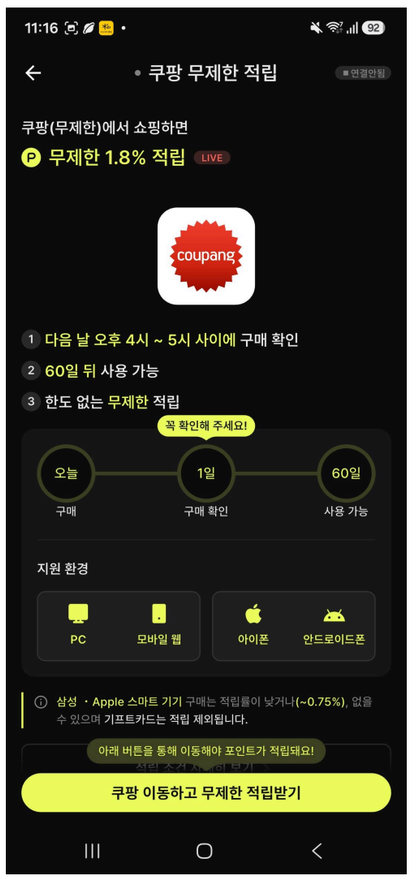
9월에 입사하자마자 외주 개발자에게 인수인계를 받아, 외주사가 6개월간 개발한 앱을 혼자 이어 개발하고 운영했습니다.



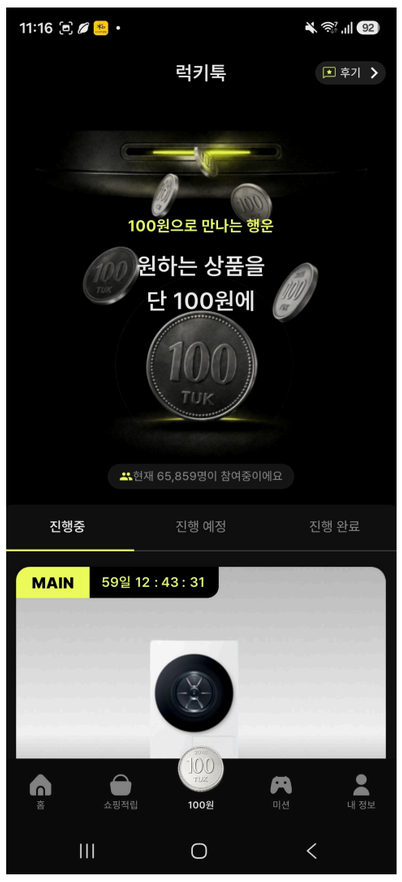
캡션 입력



캡션 입력



캡션 입력



캡션 입력

발열

갤럭시 폴드6에서만 심한 발열, 원인은 비디오 갱신

앱을 켜 두면 가을인데도 손에 쥐면 바로 느껴질 만큼 뜨거워졌지만, 갤럭시 폴드6에서만 심하게 나타나 알아채기도 재현하기도 어려웠습니다. 원인이 하나로 특정되지 않아 가장 무거울 것 같은 후보(자동 재생 비디오, 카드마다 도는 1초 타이머, 빌드마다 위젯을 새로 만드는 UniqueKey)부터 하나씩 걷어냈고, 결정적인 원인은 비디오 갱신이었습니다. 비디오는 용량을 줄여 다시 인코딩하고 화면에 보이는 상태가 바뀔 때만 재생과 정지를 처리했으며, 타이머는 전역 타이머 하나로 모으고 lint에 const 규칙을 강제해 재발을 막았습니다.

재가입 보상

탈퇴·재가입으로 반복 지급되던 가입 보상을 막고 환수

출납을 모니터링하다 이상 수치를 발견했습니다. 한 사람에게 한 번만 줘야 하는 가입 보상 500포인트가 탈퇴와 재가입을 반복하면 계속 쌓이고 있었습니다. 원인은 휴대폰 본인인증의 CI가 유지되지 않아 같은 사람을 새 사용자로 처리한 것이었고, 이를 수정해 재가입자를 식별하게 한 뒤 반복 지급된 포인트는 환수했습니다.

돌아보면

특정 기기 문제는 모니터링으로 먼저 알아채야

발열은 체감으로만 확인했고, 특정 기기에서만 심해 발견 자체가 어려웠습니다. 지금이라면 Firebase Performance Monitoring으로 기기별 성능 지표를 모으고 Android Thermal API로 읽은 발열 상태를 함께 기록해, 사용자가 먼저 느끼기 전에 어떤 기기에서 문제가 나는지 알아채겠습니다.

사용 기술

Flutter / Dart / Riverpod / flutter_hooks / Firebase

비드톡톡 play.google.com/store/apps/details?id=kr.co.newjini.bidtoktok

하나의 코드베이스로 협력사 브랜드 앱 16개 배포

뉴지니 / 2024.01 - 2025.08
Flutter 앱 구조 설계와 빌드·배포 담당

비드톡톡은 전자입찰 공고 탐색과 AI 분석을 제공하는 B2B 서비스입니다.

협력사들이 각자 자기 이름으로 컨설팅 계약을 따오는 구조라, 같은 제품이 협력사 브랜드로 배포됩니다.



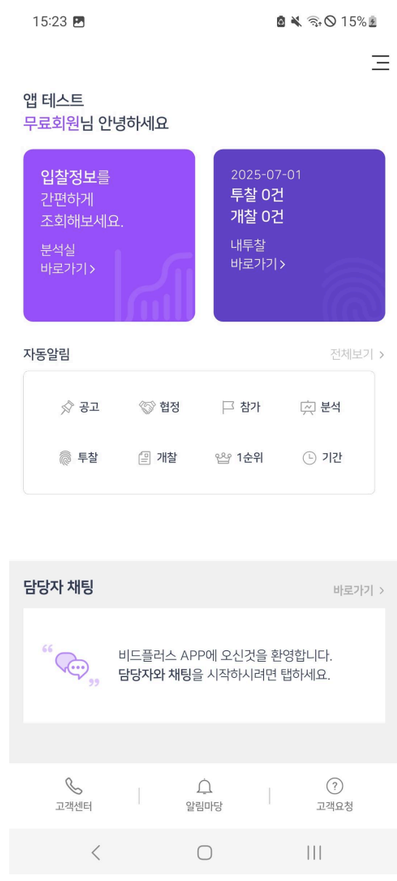
캡션 입력



캡션 입력



캡션 입력



캡션 입력

문제

계약이 늘 때마다 브랜드 앱이 하나씩 필요

브랜드마다 다른 것이 앱 이름과 아이콘만이 아니었습니다. 협력사가 각자 자기 도메인으로 운영해 API 서버 주소가 달랐고, 음성 안내 파일과 Firebase 프로젝트도 브랜드별로 따로였습니다. 코드를 복사해 앱을 따로 만들면 기능 하나를 고칠 때마다 같은 수정을 브랜드 수만큼 반복해야 했습니다.

구조

브랜드를 코드 복사가 아니라 빌드 설정으로 분리

flutter_flavorizr로 브랜드마다 Android productFlavor를 만들고, 패키지명·앱 이름·서버 주소·서명 키를 지정해 buildConfigField로 빌드 시점에 주입했습니다. 각 앱에는 자기 브랜드 설정만 들어가고, 브랜드가 늘어도 코드베이스를 복사하지 않습니다. 브랜드 추가 절차는 문서로 정리하고 런처 아이콘은 스크립트로 한 번에 생성했습니다.

운영

2주 사이 14개 추가, 이후 1인으로 16개 앱 유지

2024년 7월 구조를 세운 뒤, 그해 10월에는 2주 사이 이 구조 위에 브랜드 14개가 추가됐습니다. 협력사마다 나뉜 서명 키 3종과 브랜드별 Firebase 설정을 함께 관리했고, 알림은 FCM data 메시지로 받아 앱이 직접 구성해 종류별로 TTS 음성과 아이콘을 다르게 붙였습니다. 모바일 담당이 한 명으로 줄어든 뒤에도 16개 앱의 배포와 스토어 심사를 유지했습니다.

사용 기술

Flutter / Dart / flutter_flavorizr / Firebase / Android Gradle

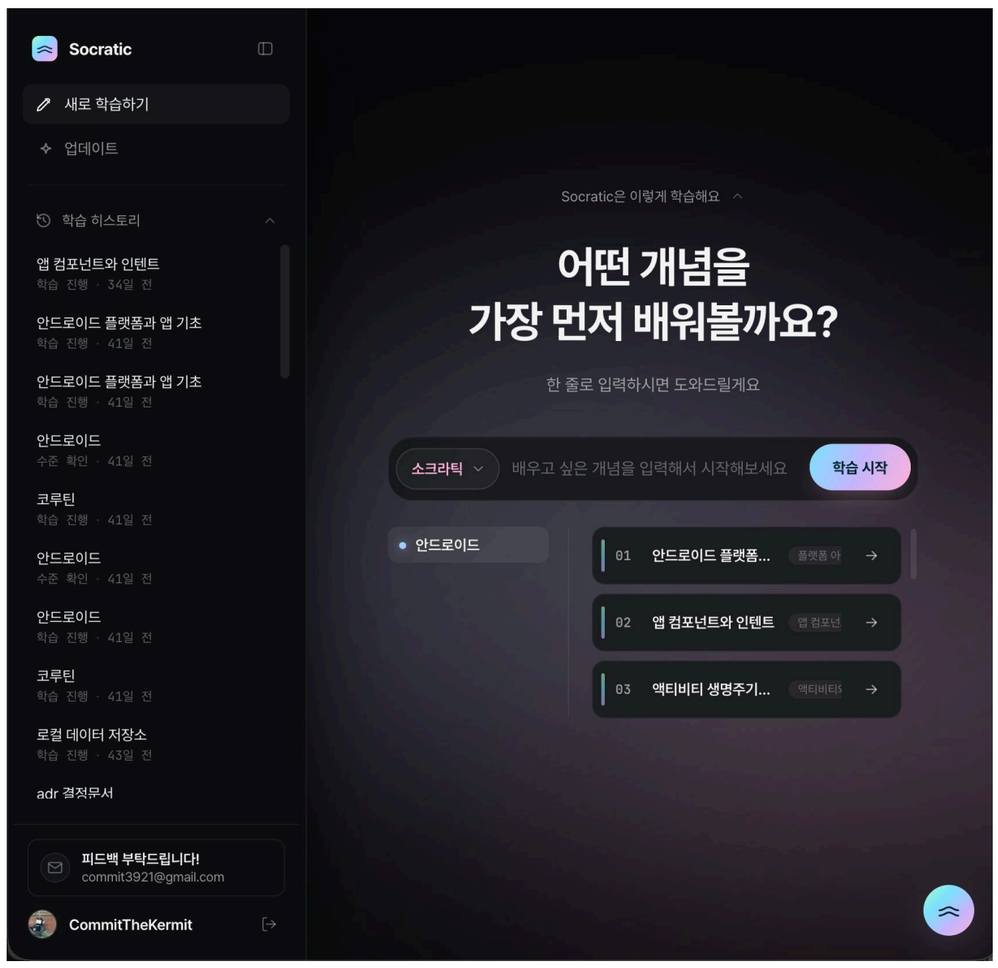
소크라틱 런 socratic-learn-web.web.app

로그인 장벽을 걷어내고 비용은 상한으로 막기

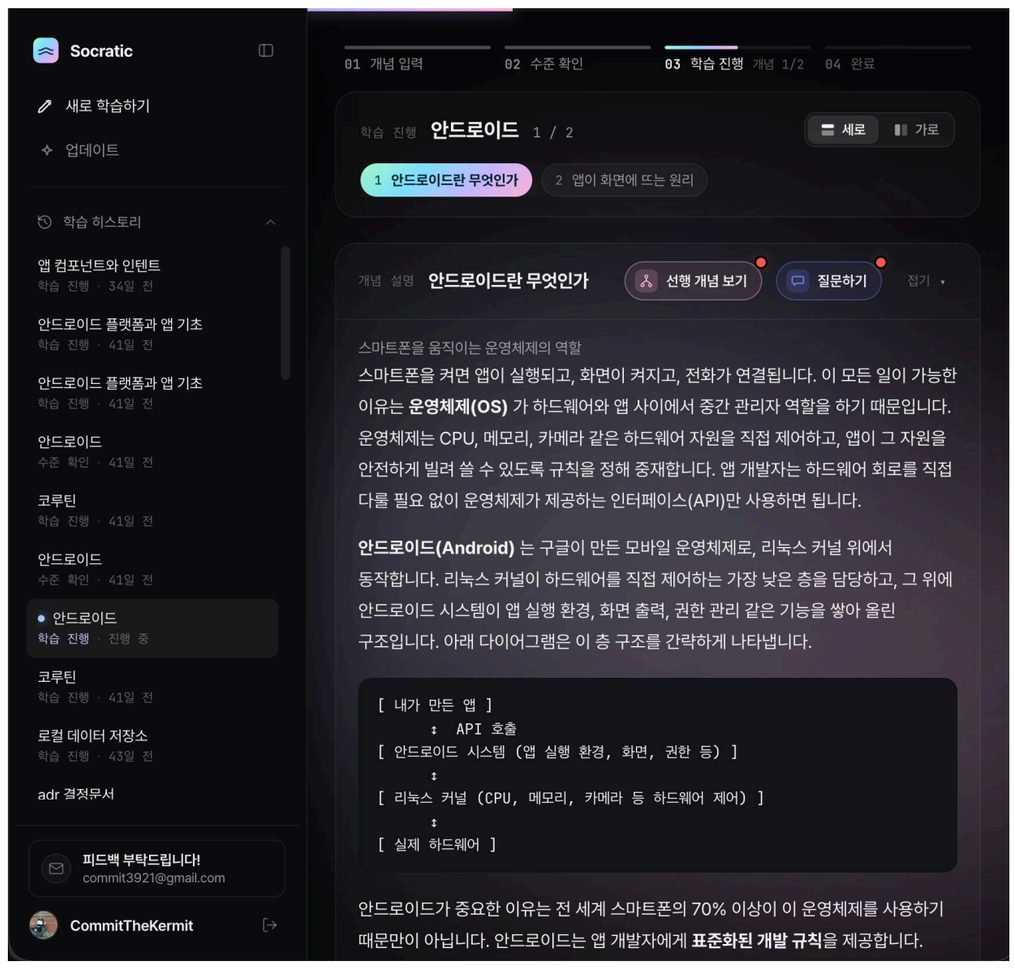
해커톤 → 개인 프로젝트 / 2026.05 - 진행 중
기획, 프론트엔드, Firebase Functions, Claude API 연동

배우고 싶은 주제를 입력하면 설명을 쏟아내는 대신 질문부터 던집니다. 진단 문항으로 아는 것과 모르는 것의 경계를 찾고, 주제를 작은 개념으로 나눠 배울 순서를 그린 다음, 한 개념씩 직접 답하며 이해를 확인합니다. 답변을 채점해 복습, 선행 학습, 다음 단계, 심화 중 지금 필요한 길을 추천합니다.

원래는 Android 코루틴을 AI와 공부하다가, AI가 답 대신 질문으로 이끄는 방식이 마음에 들어 Claude Code 스킬로 고정한 CLI 도구였습니다. 신기해하며 써 보고 싶다는 크루들이 생겨 스킬 저장소로 공유했지만, 번호를 골라 입력하는 방식이라 쓰기 번거롭고 Claude Code 사용자만 접근할 수 있었으며, 학습 기록도 남지 않았습니다. 그래서 해커톤에서 웹 서비스로 만들자고 제안해 4명이 팀으로 옮겼고, 이후 개발은 혼자 이어받았습니다.



캡션 입력



캡션 입력

문제

로그인 게이트 중 학습 시작은 11.5%

AI 호출 비용을 제가 부담해야 해서, 무제한 호출이 걱정돼 로그인한 사용자만 질문하게 막았습니다. 계산이 아니라 불안이 근거였습니다. GA4 퍼널로 17일을 보니 학습 진입은 11.5%였고, 로그인은 제가 세운 장벽이라 가장 먼저 의심할 지점이었습니다.

판단

로그인 대신 호출 상한으로 비용 방어

uid 기준 일일 150회 · 분당 20회 상한을 Anthropic 호출에만 걸고, 카운터는 트랜잭션으로 올려 동시 호출에도 새지 않게 했습니다. 저장소 장애 시에는 통과시키고 최종 방어선은 API 결제 한도에 뒀습니다. 기록 저장은 익명 인증으로 옮기고, 로그인하면 linkWithPopup으로 같은 uid를 계정으로 승격합니다.

구현

LLM이 지어내던 선택지를 코드가 구성

진단 → 로드맵 → 학습 → 분기 → 완료 흐름을 만들었습니다. '다음 단계로 이동'은 LLM이 없는 로드맵을 지어내서, 프론트가 실제 로드맵으로 선택지를 만들게 했습니다. 자주 찾는 주제는 난이도별 로드맵을 미리 두었고(Android 로드맵은 직접 작성), 주요 흐름은 단위 테스트와 Playwright로 묶었습니다.

결과

학습 진입률 11.5% → 26.7%

이후 방문자가 15명이라 확정 수치가 아닌 방향성으로 봤고, 로그인 화면 이벤트가 없어 page_view → 학습 시작을 대체 지표로 썼습니다. 막연한 불안으로 세운 방어가 진입을 막고 있었을 수 있다는 것을 숫자로 알았습니다.

사용 기술

React / TypeScript / Firebase / Claude API / Vitest / Playwright

---

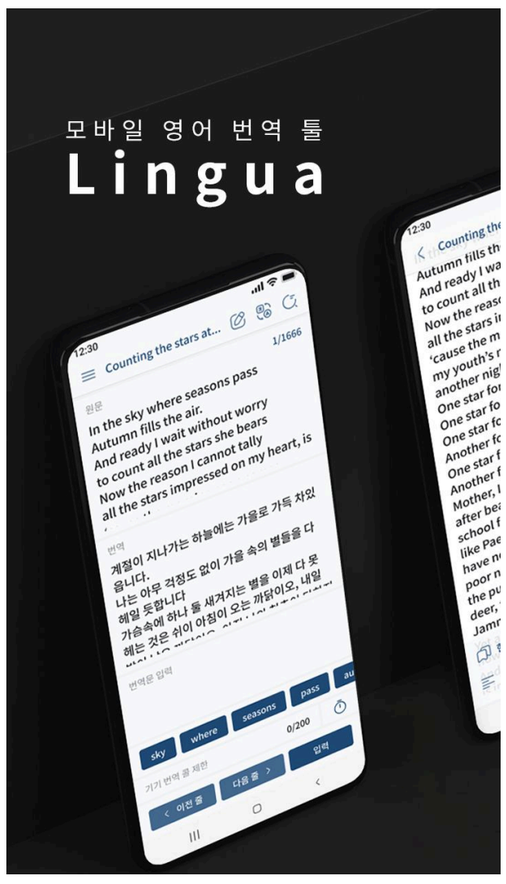
링구아 github.com/CommitTheKermit/lingua

구조와 기준은 내가 정하고, 구현은 에이전트에게 맡기기

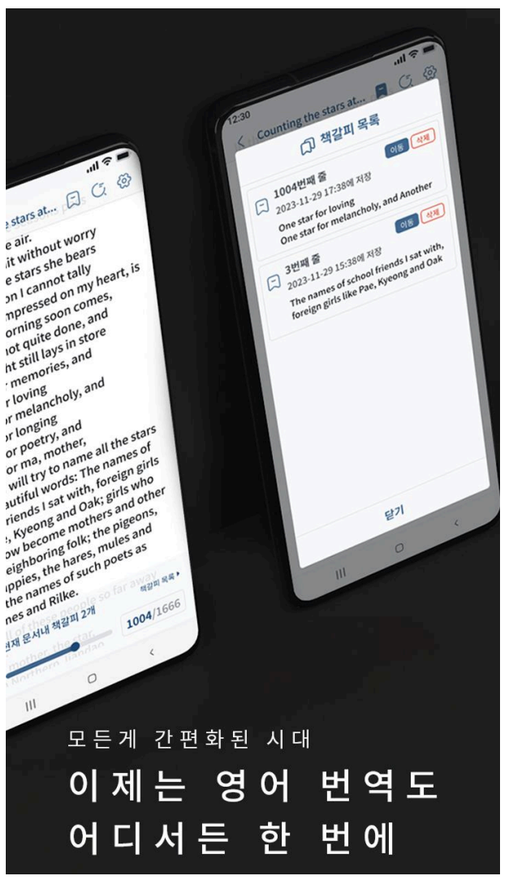
개인 프로젝트 / 2023.07 - 진행 중
Kotlin · Compose Multiplatform 재작성, 웹 베타

링구아는 영어 원서를 한 문장씩 읽으며 기계번역과 오프라인 영한 사전을 함께 보는 리더입니다. 2023년 Flutter로 만든 앱을 2026년 Kotlin · Compose Multiplatform으로 다시 만들었습니다.

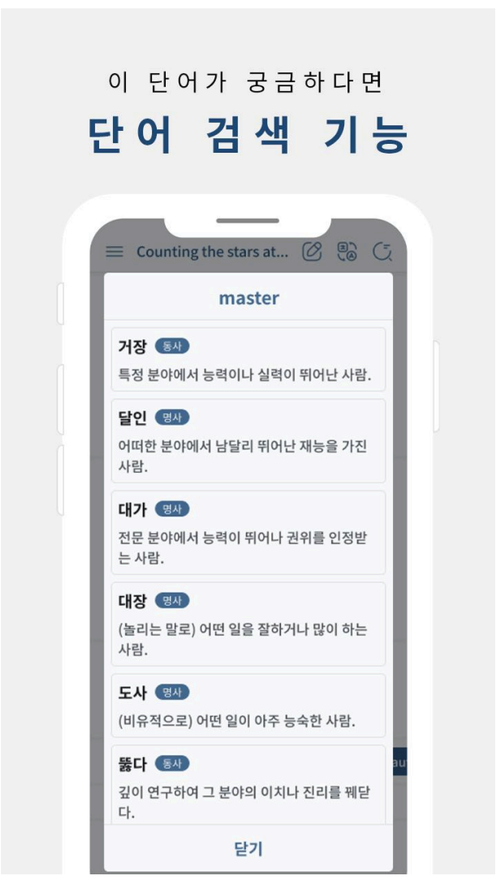
무엇을 만들지, 구조를 어떻게 나눌지, 무엇을 완료로 볼지는 제가 정했고, 구현은 Codex 에이전트에게 맡겼습니다.



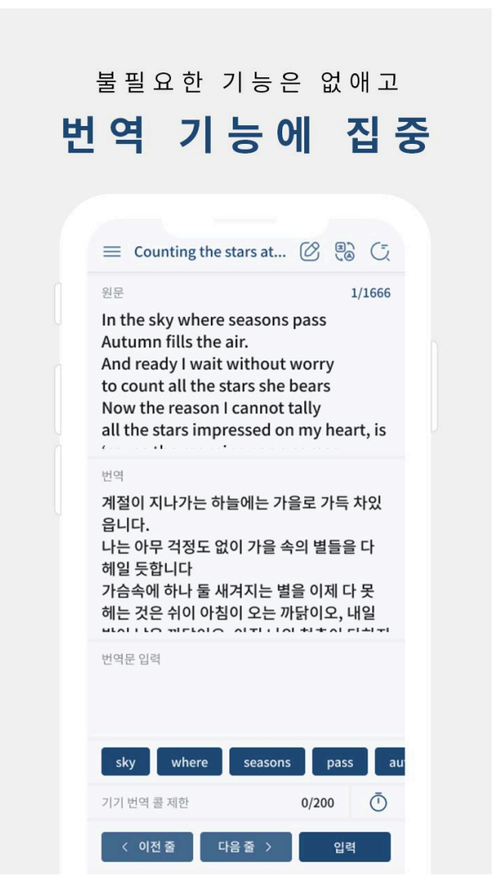
캡션 입력



캡션 입력



캡션 입력



캡션 입력

구조

옮기지 않을 것부터 문서로 고정

Flutter 앱은 ApiUtil 한 클래스가 사전 조회, 기록, DeepL 호출, 비밀값 취득을 모두 맡았고 앱이 번역 키를 직접 받아 썼습니다. 목표 구조와 옮기지 않을 것을 AGENTS.md와 계획 문서에 먼저 적어 에이전트가 따르게 했고, 플랫폼별 코드는 SQLite 드라이버와 DB 설치 두 곳으로 제한했습니다. 번역 키는 서버에만 두고 사전은 오프라인 SQLite로 분리했습니다.

범위

에이전트가 뺀 문장 번역을 되돌림

에이전트가 범위를 줄이며 문장 전체 기계번역을 제외했지만, 사용자는 원문과 기계번역을 함께 원한다고 보고 되돌렸습니다. 단어는 오프라인 사전, 문장은 서버 번역으로 나눠 제공합니다.

검증

"완료"를 피그마 화면 목록으로 대조

에이전트는 전환을 마쳤다고 했지만 실행해 보니 UI부터 달랐습니다. 피그마 화면 목록을 SSOT로 정하고 Flutter 코드는 참고로만 쓰도록 기준을 다시 세워 화면 단위로 누락을 찾았고, 피그마에 없던 상태는 피그마 다이얼로그 컴포넌트로 구성했습니다.

방향

앱이 꼭 필요한지 다시 묻고 웹으로 전환

웹과 앱을 모두 만들고 나니 앱이 꼭 필요한지 의문이 생겼습니다. 앱 설치에 대한 거부감이 크다고 보고 React 웹 베타와 DeepL 번역 서버(월별 문자 한도 · 캐시)를 구성해 새로 실험하고 있습니다.

구현하는 방식

인터뷰로 확정하고, 쪼개서 병렬로 맡기기

요구사항은 인터뷰 형식(ooo interview)으로 먼저 확정하고, 작업을 작은 완료 조건 단위로 쪼개 워크트리에서 병렬로 실행했습니다. 결과는 패리티 감사 문서와 에뮬레이터 대조 체크리스트로 확인했습니다. 맡길수록 무엇을 만들지보다 무엇을 완료로 볼지를 먼저 정해야 했습니다.

사용 기술

Kotlin / Compose Multiplatform / Firebase Functions / SQLite / React / Python / DeepL API